



StackMaster Tech

CLOUD · DATA · OBSERVABILITY · SRE

FIELD HANDBOOK · OBSERVABILITY & SIEM TRACK

Splunk Search & Event Analysis

A complete, hands-on reference to exploring events, fields, search commands and the full library of **eval** functions — with flowcharts, diagrams, SPL commands and a working dataset.

Retrieving Events

Time Search

stats · eventstats · streamstats

fieldsummary · appendpipe

eval Functions ×8

End-to-End Dataset

Vivek Arora

Corporate Trainer, Author & Consultant
StackMaster Tech · Bengaluru, India

youtube.com/@StackMasterTech
github.com/vivek9325
linkedin.com/in/vivek11arora

Contents

GETTING STARTED

How Splunk Search Works — the Search Pipeline	01
The Working Dataset — Buttercup Games Access & Security Logs	02

PART 1 · EVENTS, EVENT DATA, FIELDS & COMMANDS

1 · About Retrieving Events	03
2 · About Searching with Time	04
3 · Search Command Primer — Wildcards, Expressions, CASE	05
4 · Statistical Analysis with the stats Command	06
5 · Using fieldsummary	07
6 · Using appendpipe	08
7 · Using eventstats	09
8 · Using streamstats	10

PART 2 · EXPLORING EVAL COMMAND FUNCTIONS

9 · The eval Command — Foundations	11
10 · Conversion Functions	12
11 · Date & Time Functions	13
12 · String Functions	14
13 · Comparison & Conditional Functions	15
14 · Informational Functions	16
15 · Statistical Functions	17
16 · Mathematical Functions	18
17 · Cryptographic Functions	19

APPENDICES

A · Command Decision Flowchart — stats vs eventstats vs streamstats	20
B · SPL Quick-Reference Cheat Sheet	21

01 · How Splunk Search Works — the Search Pipeline

Every result you ever see in Splunk is produced by a **search pipeline**: a chain of processing stages joined by the pipe character (`|`). Data flows left to right — the output of one command becomes the input of the next. Understanding this flow is the single most important mental model in SPL, because it explains *why* commands behave the way they do and where each one is allowed to sit in a search.

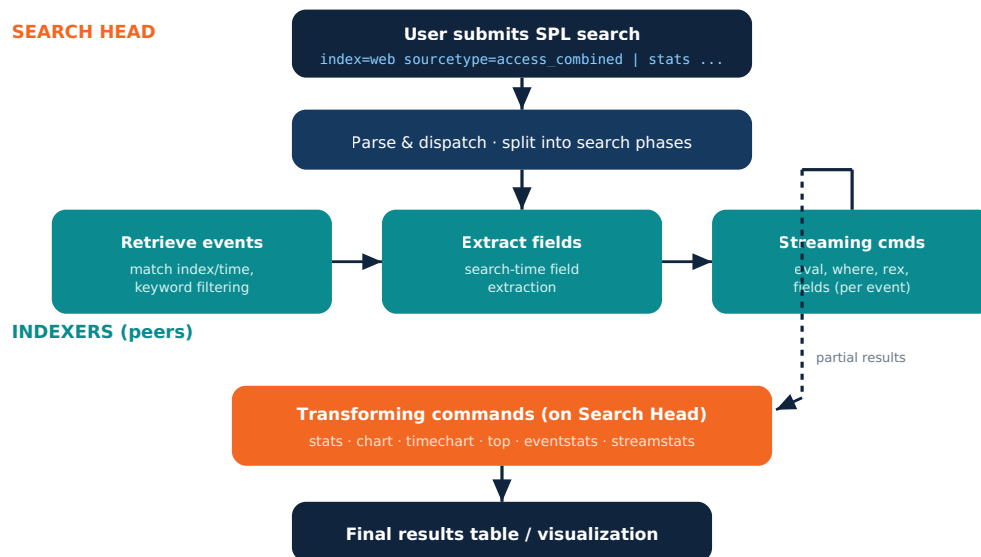


Fig 1.1 — The search pipeline. Filtering and per-event work is distributed to indexers; aggregation happens on the search head.

Streaming vs. Transforming commands

SPL commands fall into two broad families, and knowing which is which tells you where a command runs and how it scales.

Family	What it does	Examples
STREAMING	Operates on each event independently, one at a time. Can run distributed on the indexers.	<code>eval, where, rex, fields, rename, search, regex, streamstats*</code>
TRANSFORMING	Orders results into a data table, aggregating many events into summary rows. Runs on the search head.	<code>stats, chart, timechart, top, rare, table, fieldsummary</code>

NOTE

`streamstats` and `eventstats` are *dataset-processing* commands: they compute aggregates but

KEEP EVERY EVENT

instead of collapsing them, so they behave like a hybrid. We cover both in depth in Part 1.

GOLDEN RULE OF PERFORMANCE

Filter early, aggregate late. Put your most restrictive terms (index, sourcetype, time, keywords) as far left as possible so indexers discard non-matching events before any expensive command runs.

02 • The Working Dataset

Every example in this handbook runs against one consistent, realistic dataset so you can follow the story end-to-end. It models a fictional online store, **Buttercup Games** — the same shape as Splunk's official tutorial data — plus a small stream of authentication/security events. Two indexes are used throughout.

INDEX: WEB

Apache-style access logs.

sourcetype = `access_combined`

INDEX: SECURITY

Linux auth & firewall events.

sourcetype = `linux_secure`

Sample raw event WEB / ACCESS_COMBINED

```
182.236.164.11 - - [18/Jul/2026:09:14:22 +0000] "GET /product.screen?productId=SG-G01&action=purchase&categoryId=STRATEGY&price=24.99&JSESSIONID=SD1SL7FF6ADFF4953 HTTP/1.1" 200 3467 "http://buttercupgames.com/cart.do" "Mozilla/5.0 (Windows NT 10.0)" 431
```

WHERE THE FIELDS COME FROM

The URL query string carries `action`, `productId`, `categoryId`, `price` and `JSESSIONID` as `key=value` pairs, so Splunk auto-extracts them at search time. `clientip`, `status`, `bytes`, `referer` and `useragent` come from the built-in `access_combined` parsing; `product_name` is resolved from `productId` via a lookup.

Fields available after extraction

Field	Type	Example values	Meaning
<code>_time</code>	timestamp	2026-07-18 09:14:22	Event time (indexed field)
<code>clientip</code>	string	182.236.164.11	Visitor IP address
<code>action</code>	string	purchase, addtocart, view, remove	User action on the store
<code>product_name</code>	string	Mediocre Kingdoms, Dream Crusher	Game title
<code>categoryId</code>	string	STRATEGY, ARCADE, SHOOTER, TEE	Product category
<code>productId</code>	string	SG-G01, WC-SH-A01	SKU / product code
<code>price</code>	number	24.99, 39.99	Unit price (USD)
<code>status</code>	number	200, 404, 500, 503	HTTP response code
<code>bytes</code>	number	3467, 1290	Response size in bytes
<code>clientip · JSESSIONID</code>	string	SD1SL7FF6ADFF4953	Session identifier
<code>useragent</code>	string	Mozilla/5.0 (Windows NT 10.0)	Browser string
<code>referer</code>	string	http://buttercupgames.com/cart.do	Referring URL

Product catalogue LOOKUP • PRODUCTS.CSV

Eight titles across four categories drive the store. `product_name` and the canonical `price` resolve from `productId` through the `products.csv` lookup shipped with the dataset.

productId	product_name	categoryId	price
SG-G01	Mediocre Kingdoms	STRATEGY	24.99
PZ-SG-G05	SIM Cubicle	STRATEGY	19.99
DB-SG-G01	Benign Space Debris	ARCADE	24.99
FS-SG-G03	Final Sequel	ARCADE	14.99
WC-SH-A01	Dream Crusher	SHOOTER	39.99
WC-SH-A02	Orvil the Wolverine	SHOOTER	29.99
WC-SH-T02	World of Cheese Tee	TEE	9.99
MB-AG-T01	Manganiello Bros. Tee	TEE	11.99

Raw events in the Splunk search app

index=web sourcetype=access_combined action=purchase Last 7 days

Events Patterns Statistics Visualization

6,000 events (7/11/26 to 7/18/26) · job ██████████ 100%

7/18/26 09:00:00	121.219.156.140 - - [11/Jul/2026:09:00:00 +0000] "GET /product.screen?productId=FS-SG-G03&action=view&categoryId=ARCADE&price=14.99&SESSIONID=SDT8FI4BBX0FA73B HTTP/1.1" 200 2562 "http://buttercupgames.com/cart.do" "Mozilla/5.0 (X11; Linux x86_64)" 854
7/18/26 09:01:00	152.244.139.174 - - [11/Jul/2026:09:01:00 +0000] "GET /product.screen?productId=DB-SG-G01&action=view&categoryId=ARCADE&price=24.99&SESSIONID=SD3RM21AVJ6KH2GD HTTP/1.1" 200 611 "https://www.google.com/" "Mozilla/5.0 (iPhone; CPU iPhone OS 17_5 like Mac OS X)" 425
7/18/26 09:05:12	77.91.40.201 - - [11/Jul/2026:09:05:12 +0000] "GET /product.screen?productId=WC-SH-A01&action=view&categoryId=SHOOTER&price=39.99&SESSIONID=SD07PUA9H72T5E59 HTTP/1.1" 304 0 "http://buttercupgames.com/cart.do" "Mozilla/5.0 (iPhone; CPU iPhone OS 17_5 like Mac OS X)" 190

Fig 2.1 — index=web sourcetype=access_combined action=purchase in the Events tab. Auto-extracted key=value fields are highlighted.

Sample raw event SECURITY / LINUX_SECURE

```
Jul 18 09:15:04 web01 sshd[4021]: Failed password for invalid user admin from 203.0.113.7 port 52344 ssh2
```

Field	Example	Meaning
host	web01	Server that produced the event
user	admin	Account referenced in the auth attempt
src_ip	203.0.113.7	Source IP of the connection
vendor_action	Failed password / Accepted password	Outcome of the authentication
port	52344	Source port

HOW TO LOAD IT

This handbook ships with a companion dataset package ([Buttercup_Games_Dataset.zip](#)). Copy the `stackmaster_buttercup` app into `$SPLUNK_HOME/etc/apps` and restart — it creates the `web` and `security`

indexes, all field extractions, and the product lookup. Then upload `access_combined.log` (sourcetype `access_combined`, index `web`) and `linux_secure.log` (sourcetype `linux_secure`, index `security`) via **SETTINGS → ADD DATA → UPLOAD**. Timestamps span the last 7 days, so every `earliest=-7d@d` example returns data immediately. Every SPL block in this book is copy-paste runnable against it.

PART ONE

Events, Event Data, Fields & Commands

How Splunk finds events, how time bounds every search, and how the core analytic commands — **stats**, **fieldsummary**, **appendpipe**, **eventstats** and **streamstats** — turn raw events into answers.

03 · About Retrieving Events

The first command in almost every search is an implicit `search`. Its job is **retrieval**: telling the indexers which slice of data to pull off disk. The more precisely you scope retrieval, the faster and cheaper everything downstream becomes.

The four retrieval selectors

Four metadata fields are indexed at ingest and are the fastest possible filters. Lead with them.

Selector	Purpose	Example
<code>index</code>	Which data repository to read. Always specify it — the default is usually just <code>main</code> .	<code>index=web</code>
<code>sourcetype</code>	The data format / parsing rules. Distinguishes log shapes.	<code>sourcetype=access_combined</code>
<code>source</code>	The specific file, stream, or input path.	<code>source="/var/log/secure"</code>
<code>host</code>	The machine that generated the event.	<code>host=web01</code>

Basic retrieval patterns

```
# Everything in the web index (broad – avoid in production)
index=web

# Scoped by index + sourcetype – the standard opening line
index=web sourcetype=access_combined

# Add a keyword filter: only events containing the term "purchase"
index=web sourcetype=access_combined purchase

# Field-value filter: only successful purchases
index=web action=purchase status=200
```

KEYWORD VS. FIELD-VALUE

A bare term like `purchase` matches the raw text of the event (fast, indexed). A field expression like `action=purchase` matches an extracted field (precise). Combine both: keyword to narrow at index time, field to be exact.

Controlling how many events come back

Command	Effect
<code> head 20</code>	Keep the first 20 events returned (most recent, by default reverse-time order).
<code> tail 20</code>	Keep the last 20 events (oldest of the set).
<code> fields clientip, action, price</code>	Keep only the listed fields — reduces data carried through the pipeline.
<code> fields - _raw</code>	Drop a field (here the bulky <code>_raw</code>) to speed up later commands.
<code> dedup clientip</code>	Keep only the first event per unique <code>clientip</code> .

Inspecting what exists before you search

Two commands read the index catalog itself, so you can discover data without scanning events.


```
# List indexed metadata values and their event counts
| metadata type=sourcetypes index=web

# Tabulate counts across metadata dimensions (very fast, catalog-only)
index=web | tstats count WHERE index=web BY sourcetype
```

NOTE

`tstats` works only against indexed/tsidx fields but is dramatically faster than `stats` for those, because it never opens the raw events. Reach for it when counting by `index`, `sourcetype`, `source` or `host`.

04 · About Searching with Time

Time is not just a filter in Splunk — it is the primary index. Every event is stored by its `_time` value, so restricting the time range is the cheapest and most powerful way to limit a search. You set time three ways: the **time-range picker** in the UI, inline `earliest` / `latest` modifiers, or relative-time expressions.

Relative time syntax

A relative time string has up to three parts: an **offset**, a **unit**, and an optional **snap**.

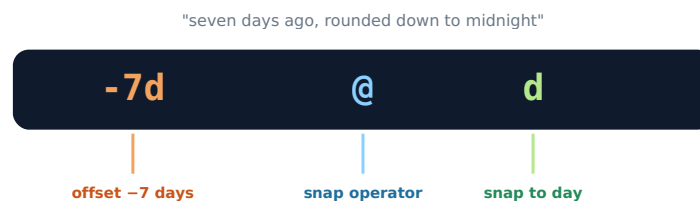


Fig 4.1 — Anatomy of the relative-time modifier `-7d@d`.

Unit	Abbreviations	Example
Second	s, sec, secs, second	-30s → 30 seconds ago
Minute	m, min, minute	-15m → 15 minutes ago
Hour	h, hr, hour	-4h → 4 hours ago
Day	d, day	-1d → yesterday, this time
Week	w, week, w0=Sun...w6=Sat	-1w → one week ago
Month	mon, month	-1mon → one month ago
Quarter / Year	q, qtr / y, yr, year	-1y@y → start of last year

The snap operator (@)

The `@` "snaps" a time to the nearest earlier boundary of a unit — a rounding-down operation. This is what makes ranges align to clean calendar boundaries.

Expression	Resolves to (if now = Sat 18 Jul 2026, 09:14:22)
@d	Today at 00:00:00 (snap to start of day)
@h	09:00:00 (snap to top of the hour)
@w0	Start of the week, Sunday 00:00
@mon	1 Jul 2026, 00:00 (start of month)
-1d@d	Yesterday 00:00 — snap first, then the offset already applied
@d-1h	Today 00:00 minus one hour = yesterday 23:00

Inline earliest / latest

```
# Last 7 whole days, up to now
index=web earliest=-7d@d latest=now

# Yesterday only (a clean 24h window)
index=web earliest=-1d@d latest=@
```

```
# A precise absolute window
index=web earliest="07/17/2026:00:00:00" latest="07/18/2026:00:00:00"
```

COMMON TRAP

`earliest=-24h` and `earliest=-1d@d` are

NOT

the same. The first is a rolling 24-hour window ending now; the second is the fixed calendar day. Use snapping whenever you want repeatable, dashboard-friendly ranges.

Grouping events over time with timechart

Once time is bounded, `timechart` buckets events into equal spans for trend analysis.

```
index=web sourcetype=access_combined earliest=-24h@h
| timechart span=1h count BY action
```

TIP — BIN/BUCKET

To group by time *without* charting, use `| bin _time span=1h` then `stats`. `timechart` is really `bin _time` + `stats` fused together.

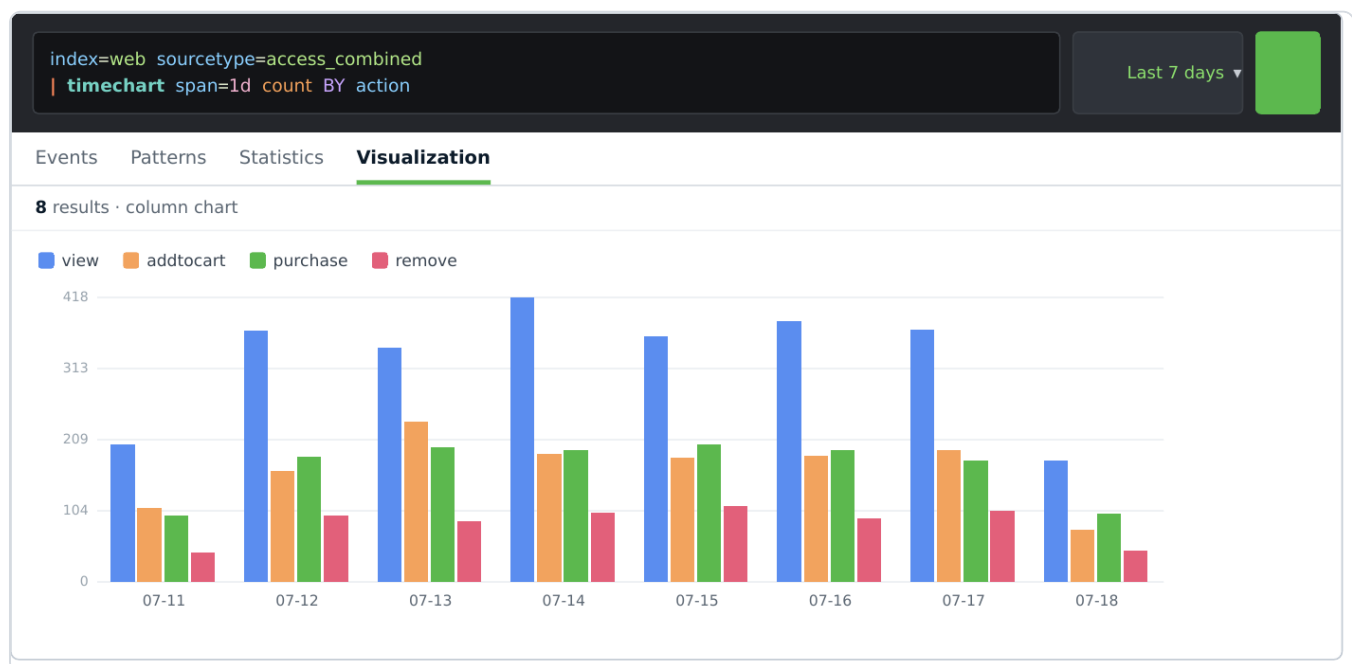


Fig 4.2 — timechart span=1d count BY action over the dataset's 7-day window (Visualization tab).

05 · Search Command Primer — Wildcards, Expressions, CASE

The `search` command is the workhorse filter. It appears implicitly at the start of every search and can be reused anywhere in the pipeline to narrow results further. Mastering its expression syntax lets you say precisely what you mean.

Boolean logic

Operators **must be uppercase**: `AND`, `OR`, `NOT`. Whitespace between terms is an implicit `AND`. Use parentheses to control precedence.

```
# Implicit AND – both terms must appear
index=web purchase status=200

# OR with grouping
index=web (action=purchase OR action=addtocart) status=200

# NOT vs != – NOT excludes events missing the field too
index=web NOT status=200
```

NOT VS !=

`status!=200` keeps only events that *have* a status field whose value is not 200. `NOT status=200` also keeps events where `status` is absent entirely. The difference bites when fields are sparse.

Comparison operators (field expressions)

Operator	Meaning	Example
<code>= / !=</code>	Equal / not equal	<code>action=purchase</code>
<code>< <= > >=</code>	Numeric / lexical comparison	<code>price>=25</code>
<code>field=*</code>	Field exists (has any value)	<code>product_name=*</code>
<code>field=""</code>	Field is empty / null	<code>referer=""</code>

Wildcards

The asterisk `*` matches any number of characters. It is powerful but can be slow when leading, because a leading wildcard defeats the index.

```
# Trailing wildcard (efficient): any 5xx server error
index=web status=5*

# Match product families
index=web productId=WC-SH-*

# Leading wildcard (AVOID – cannot use the index)
index=web referer=*checkout*
```

PERFORMANCE

Prefer `term*` over `*term`. If you truly need "contains", consider extracting a field and matching it with `like()` or `match()` in `where` instead.

Phrases and quoting

```
# Quote multi-word phrases and values with spaces/special chars
index=web "GET /product.screen"
index=security vendor_action="Failed password"

# Escape a literal quote or backslash with a backslash
index=web "user said \"hello\""
```

Search directives: CASE() and TERM()

By default Splunk keyword matching is **case-insensitive**. Two directives change that behaviour for a single term.

Directive	Behaviour
CASE(x)	Forces a case-sensitive match of the term <code>x</code> . <code>CASE(Error)</code> matches "Error" but not "error" or "ERROR".
TERM(x)	Treats <code>x</code> as a single indexed term , ignoring minor breakers (like <code>.</code> <code>-</code> <code>/</code>) inside it. Great for matching IPs, GUIDs, or paths exactly and fast.

```
# Case-sensitive: only capital-F "Failed"
index=security CASE(Failed)

# Match the whole IP as one term (faster than clientip=... for raw text)
index=web TERM(182.236.164.11)

# TERM shines on values containing minor breakers
index=web TERM(SD1SL7FF6ADFF4953)
```

WHEN TO USE TERM()

If a value contains characters Splunk treats as breakers and you want an exact, index-accelerated match without wildcards, wrap it in `TERM()`. It often turns a slow wildcard search into a fast one.

Filtering later in the pipeline: search vs where

Command	Use when...
search	You are matching keywords, wildcards, or field=value comparisons — same syntax as the base search.
where	You need an expression : comparing two fields, using eval functions, or math. e.g. <code> where bytes > avg_bytes</code>

```
# search: keyword/field filter
... | search action=purchase price>20

# where: field-to-field and function-based comparison
... | where price > avg_price AND like(productId, "WC-%")
```

06 • Statistical Analysis with the stats Command

`stats` is the most important transforming command in SPL. It collapses many events into a compact results table by applying **aggregate functions**, optionally grouped by one or more fields with `BY`. If you learn one command deeply, make it this one.

```
# Anatomy
| stats function(field) AS alias ... BY split_field ...
```

Aggregate function reference

Function	Returns
count(X)	Number of events (or non-null occurrences of X)
dc(X) / distinct_count(X)	Count of distinct values of X
sum(X)	Sum of numeric values
avg(X)	Arithmetic mean
median(X)	Middle value
mode(X)	Most frequent value
min(X) / max(X)	Smallest / largest value
range(X)	max(X) – min(X)
stdev(X) / stdevp(X)	Sample / population standard deviation
var(X) / varp(X)	Sample / population variance
perc95(X) / p95(X)	The Nth percentile (any N, e.g. perc50, perc99)
first(X) / last(X)	First / last value in input order
earliest(X) / latest(X)	Value with smallest / largest_time
list(X)	All values as a multivalue list (keeps duplicates, order)
values(X)	All <i>distinct</i> values, lexicographically sorted
sumsq(X)	Sum of squares (used in variance math)

Worked examples on the dataset

```
# 1) Total revenue and order count per product category
index=web action=purchase
| stats sum(price) AS revenue, count AS orders, dc(clientip) AS buyers
  BY categoryId
| sort - revenue
```

```
index=web action=purchase
| stats sum(price) AS revenue, count AS orders, dc(clientip) AS buyers BY categoryid | sort - revenue
```

Last 7 days

Events	Patterns	Statistics	Visualization
4 results			
categoryid	revenue	orders	buyers
SHOOTER	12,856.37	363	99
STRATEGY	7,711.59	341	97
ARCADE	6,646.66	334	92
TEE	3,470.88	312	94
Total	30,685.50	1350	

Fig 6.1 — Result of example 1 on the live dataset. SHOOTER leads on revenue; the total row is added with appendpipe (Ch. 8).

```
# 2) Latency-style distribution of response size per status code
index=web sourcetype=access_combined
| stats avg(bytes) AS avg_b, perc95(bytes) AS p95_b, max(bytes) AS max_b
  BY status
```

```
# 3) Multiple split-by fields + values() to see which products each buyer bought
index=web action=purchase
| stats count, values(product_name) AS products BY clientip
```

count vs count(field)

SUBTLE BUT VITAL

`count` counts every event in the group. `count(price)` counts only events where `price` is present and non-null. Use the field form to measure coverage/completeness of a field.

Renaming with AS and chaining

Every function can be aliased with `AS`. Well-named output fields make downstream `eval`, `where`, and charts readable. You can combine `stats` results with further math:

```
index=web action=purchase
| stats sum(price) AS revenue, count AS orders BY categoryId
| eval avg_order_value = round(revenue/orders, 2)
| sort - avg_order_value
```

07 · Using fieldsummary

`fieldsummary` is your data-profiling command. It produces a single summary row **per field**, describing how many events have that field, how many distinct values it holds, and — for numeric fields — its min, max, mean and standard deviation. It is the fastest way to understand an unfamiliar sourcetype.

```
# Syntax
| fieldsummary [maxvals=int] [field-list]
```

Output columns

Column	Meaning
field	The field name being summarized
count	Number of events in which the field appears
distinct_count	Number of unique values
is_exact	1 if distinct_count is exact, 0 if estimated (capped by maxvals)
numeric_count	How many of the values were numeric
min · max · mean · stdev	Numeric statistics (blank for non-numeric fields)
values	A multivalue sample of value:count pairs (limited by <code>maxvals</code>)

Examples

```
# Profile every field in the access logs
index=web sourcetype=access_combined
| fieldsummary
```

```
# Profile only chosen fields, sampling up to 10 values each
index=web sourcetype=access_combined
| fieldsummary maxvals=10 action status price
```

```
# Find sparse / low-quality fields: those present in few events
index=web sourcetype=access_combined
| fieldsummary
| eval coverage_pct = round(count / distinct_count * 100, 1)
| table field count distinct_count mean stdev
| sort count
```


index=web sourcetype=access_combined | fieldsummary

Last 7 days

Events Patterns **Statistics** Visualization

6 results

field	count	distinct_count	min	max	mean
action	6,000	4			
categoryld	6,000	4			
status	6,000	5	200.0	503.0	227.81
price	6,000	7	9.99	39.99	22.23
bytes	6,000	3,149	0.0	5200.0	2571.14
clientip	6,000	100			

Fig 7.1 — fieldsummary profiling the access logs: counts, cardinality and numeric stats per field.

WORKFLOW TIP

Run `fieldsummary` as the first thing you do on a new data source. In 5 seconds you learn which fields are numeric, which are high-cardinality (good keys), and which are nearly empty (skip them).

08 • Using appendpipe

`appendpipe` runs a sub-pipeline against **the results produced so far** and appends its output to the bottom of those same results. Unlike `append`, it does not launch a second search of the raw data — it reuses the current result set, which makes it efficient and ideal for adding **subtotals and grand-total rows**.

```
# Syntax
| appendpipe [run_in_preview=bool] [ <subpipeline> ]
```

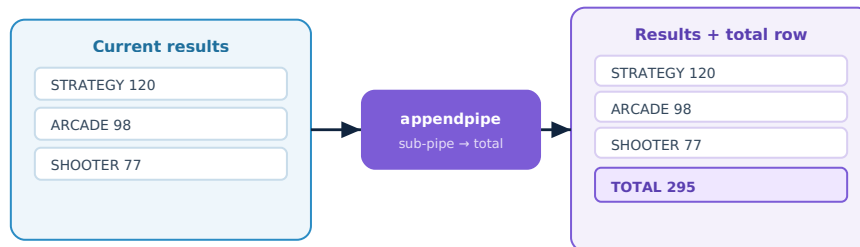


Fig 8.1 — `appendpipe` re-processes the current results and appends a summary row.

Adding a grand-total row

```
index=web action=purchase
| stats count AS orders, sum(price) AS revenue BY categoryId
| appendpipe [
  | stats sum(orders) AS orders, sum(revenue) AS revenue
  | eval categoryId = "- TOTAL -"
]
```

Adding per-group subtotals

```
index=web action=purchase
| stats sum(price) AS revenue BY categoryId product_name
| appendpipe [
  | stats sum(revenue) AS revenue BY categoryId
  | eval product_name = "Subtotal"
]
| sort categoryId product_name
```

WATCH OUT

The sub-pipeline sees only fields that exist in the current results, not the original raw events. If you need a field from the raw data, compute it *before* the `appendpipe`. Also add `| sort` afterward, since the appended rows land at the bottom unordered.

APPEND VS APPENDPIPE

`append [ subsearch ]` runs a brand-new search against the index. `appendpipe` reuses results already in the pipeline — cheaper, and the right tool for totals.

09 • Using eventstats

`eventstats` computes the same aggregates as `stats` but **does not collapse events**. Instead it glues the aggregate value onto **every** event as a new field. This lets each event "see" a group-level number — perfect for comparing an individual event to its group average, or computing a percentage of total.

```
# Syntax
| eventstats [allnum=bool] agg(field) [AS alias] ... [BY field-list]
```

STATS — COLLAPSES

```
stats avg(price) BY categoryId
# → 1 row per category, events gone
```

EVENTSTATS — ENRICHES

```
eventstats avg(price) BY categoryId
# → all events kept + avg_price added
```

Example: flag above-average-priced purchases

```
index=web action=purchase
| eventstats avg(price) AS cat_avg BY categoryId
| eval vs_avg = round(price - cat_avg, 2)
| where price > cat_avg
| table clientip product_name categoryId price cat_avg vs_avg
```

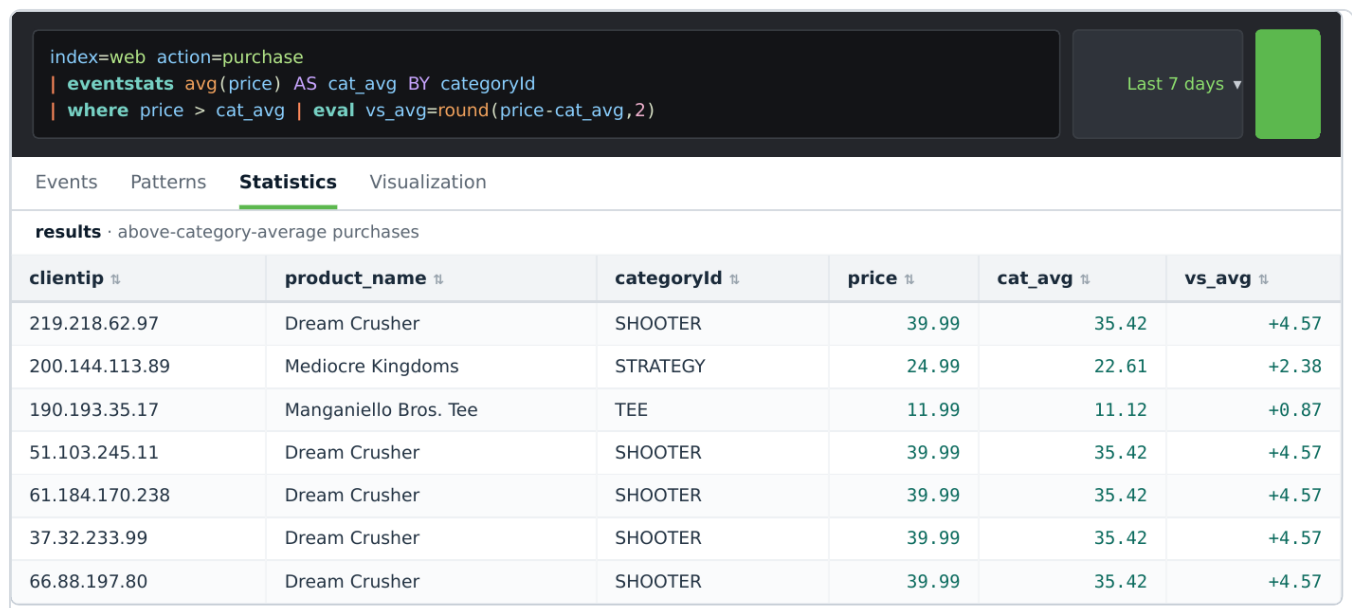


Fig 9.1 — Each purchase keeps its detail and gains `cat_avg`; `vs_avg` shows how far above its category mean it sits.

Example: each event's share of total revenue

```
index=web action=purchase
| eventstats sum(price) AS grand_total
| eval pct_of_total = round(price / grand_total * 100, 2)
```

MENTAL MODEL

Use `eventstats` whenever a later `eval` or `where` needs a group-level number *and* the original per-event detail at the same time. `stats` throws the detail away; `eventstats` keeps it.

COST

`eventstats` holds the whole result set in memory to backfill every event. On very large datasets, filter first or restrict the `BY` cardinality.

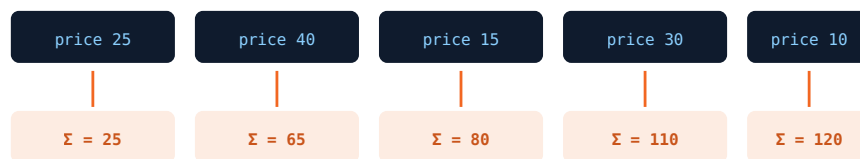
10 · Using streamstats

`streamstats` computes aggregates **cumulatively, event by event, in order**. As it walks the stream, each event receives the running result computed from itself and all events before it. This is the command for running totals, moving averages, sequence numbering, and comparing an event to the one before it.

```
# Syntax (key options)
| streamstats [current=bool] [window=int] [time_window=span]
             [reset_on_change=...] [reset_after=...] [global=bool]
             agg(field) [AS alias] ... [BY field-list]
```

Option	Effect
current	<code>true</code> (default) includes the current event in its own calculation; <code>false</code> uses only prior events.
window=N	Limit the calculation to a sliding window of the last N events (moving average).
time_window=span	Window bounded by time (e.g. <code>5m</code>) rather than event count. Requires time order.
reset_on_change	Restart the running calc when a <code>BY</code> field value changes.
reset_after="<eval>"	Restart after the eval condition is true.
global	With a window, controls whether the window is shared across BY groups.

Running total (cumulative sum)



Each event carries the sum of itself + all prior events

Fig 10.1 — `streamstats sum(price)` builds a running total across the ordered stream.

Example 1 — running revenue total over time

```
index=web action=purchase
| sort 0 _time
| streamstats sum(price) AS running_revenue, count AS order_no
```

```
index=web action=purchase | sort 0 _time
| streamstats sum(price) AS running_revenue, count AS order_no
```

Last 7 days

Events	Patterns	Statistics	Visualization
--------	----------	-------------------	---------------

first 8 of many results

_time	product_name	price	running_revenue	order_no
07-11 09:34:10	Dream Crusher	39.99	39.99	1
07-11 09:48:02	Orvil the Wolverine	29.99	69.98	2
07-11 09:51:58	Orvil the Wolverine	29.99	99.97	3
07-11 09:54:35	Mediocre Kingdoms	24.99	124.96	4
07-11 09:57:11	Manganiello Bros. Tee	11.99	136.95	5
07-11 10:23:33	SIM Cubicle	19.99	156.94	6
07-11 10:38:23	Dream Crusher	39.99	196.93	7
07-11 10:50:56	World of Cheese Tee	9.99	206.92	8

Fig 10.2 — *running_revenue* accumulates purchase by purchase; *order_no* numbers the ordered stream.

Example 2 — 5-event moving average of response size

```
index=web sourcetype=access_combined
| sort 0 _time
| streamstats window=5 avg(bytes) AS moving_avg_bytes
```

Example 3 — time between consecutive events per session

```
index=web sourcetype=access_combined
| sort 0 JSESSIONID _time
| streamstats current=f last(_time) AS prev_time BY JSESSIONID
| eval gap_secs = _time - prev_time
```

Example 4 — detect brute-force: failed logins in a 5-minute window

```
index=security vendor_action="Failed password"
| sort 0 _time
| streamstats time_window=5m count AS fails_5m BY src_ip
| where fails_5m >= 5
```

```
index=security vendor_action="Failed password" | sort 0 _time
| streamstats time_window=5m count AS fails_5m BY src_ip
| where fails_5m >= 5 | stats max(fails_5m) AS peak_5m BY src_ip
```

Last 7 days

Events	Patterns	Statistics	Visualization
--------	----------	-------------------	---------------

1 result · brute-force source detected

src_ip	peak_5m
203.0.113.7	14

Fig 10.3 — The dataset's planted attacker 203.0.113.7 surfaces cleanly (14 failures in 5 min); no other source crosses the threshold.

ORDER MATTERS

`streamstats` is order-dependent — always precede it with an explicit `| sort 0 <keys>` (the `0` removes the default 10 000-row limit) so results are deterministic.

RESET_ON_CHANGE

To restart a running total each time a group changes (e.g. per day), sort by the group key and pass

`reset_on_change=true` with a matching `BY`.

PART TWO

Exploring Eval Command Functions

The **eval** command is SPL's calculation engine. It carries eight families of functions — conversion, date/time, string, comparison & conditional, informational, statistical, mathematical and cryptographic — that let you transform, classify and enrich every event.

11 · The eval Command — Foundations

`eval` creates or overwrites a field by evaluating an expression on each event. Expressions combine fields, literals, operators and functions. If the expression can't be evaluated for an event (e.g. arithmetic on a non-number), the field is simply not set for that event.

```
# Syntax – one or more comma-separated assignments
| eval newfield = <expression> [, another = <expression> ]
```

Operators available inside eval

Category	Operators	Notes
Arithmetic	+ - * / %	<code>+</code> also concatenates when a side is a string
Comparison	== != < > <= >=	Return boolean; <code>==</code> for equality (not <code>=</code>)
Boolean	AND OR NOT ()	Combine conditions
Concatenation	. or +	The dot always joins strings: <code>"a"."b"</code>

A quick taste

```
index=web action=purchase
| eval price_with_tax = round(price * 1.18, 2),
    label = product_name . " (" . categoryId . ")",
    tier = if(price >= 40, "premium", "standard")
```

EVAL VS WHERE VS FIELDS

`eval` computes fields, `where` keeps/drops events using an expression, and `fields` keeps/drops columns. The same functions in this Part work inside `eval`, `where`, and `fieldformat`.

12 · Conversion Functions

Conversion functions change a value's **type or representation** — text to number, number to formatted text, or building formatted strings. Splunk stores field values as strings under the hood, so explicit conversion is often what makes sorting and math behave correctly.

Function	Description	Example → result
<code>tonumber(str, base)</code>	Converts a string to a number. Optional base (2–36, default 10).	<code>tonumber("0x1F", 16) → 31</code>
<code>tostring(X, fmt)</code>	Converts X to a string. <code>fmt</code> = <code>"hex"</code> , <code>"commas"</code> , or <code>"duration"</code> .	<code>tostring(1234567, "commas") → 1,234,567</code>
<code>printf(fmt, ...)</code>	Builds a string using C-style sprintf formatting.	<code>printf("%.2f", 3.1) → 3.10</code>

tostring format specifiers

Format	Effect	Example
<code>"commas"</code>	Thousands separators, 2 decimals	<code>"1,024.50"</code>
<code>"hex"</code>	Hexadecimal representation	<code>tostring(255, "hex") → 0xff</code>
<code>"duration"</code>	Seconds → HH:MM:SS (or D+HH:MM:SS)	<code>tostring(3661, "duration") → 01:01:01</code>

Examples on the dataset

```
# Format revenue with thousands separators for a report
index=web action=purchase
| stats sum(price) AS revenue BY categoryId
| eval revenue_fmt = "$" . tostring(round(revenue,2), "commas")
```

```
# Ensure status is numeric before comparing, and show session length as duration
index=web
| eval status_num = tonumber(status)
| stats range(_time) AS secs BY JSESSIONID
| eval session_len = tostring(secs, "duration")
```

```
# printf for aligned, formatted output
| makeresults
| eval msg = printf("%-10s qty=%03d total=$%.2f", "Widget", 7, 142.5)
# → "Widget      qty=007 total=$142.50"
```

WHY CONVERT?

Values sort lexically until they're numeric. Without `tonumber`, `"100"` sorts before `"20"`. Convert first, then `sort` or compare.

13 · Date & Time Functions

Splunk stores time as **epoch seconds** in `_time`. Date/time functions convert between epoch and human-readable strings, do relative-time math, and extract calendar parts. They are the backbone of trend analysis and business-hours logic.

Function	Description
<code>now()</code>	Current time as epoch seconds, fixed for the whole search
<code>time()</code>	Wall-clock time with microsecond precision at the moment of evaluation
<code>strftime(epoch, fmt)</code>	Formats an epoch value into a readable string
<code>strptime(str, fmt)</code>	Parses a time string into epoch seconds
<code>relative_time(epoch, modifier)</code>	Applies a relative-time modifier (e.g. <code>"-1d@d"</code>) to an epoch value

Common strftime / strptime directives

Code	Means	Code	Means
<code>%Y</code>	4-digit year (2026)	<code>%H</code>	Hour 00-23
<code>%m</code>	Month 01-12	<code>%M</code>	Minute 00-59
<code>%d</code>	Day of month 01-31	<code>%S</code>	Second 00-59
<code>%b / %B</code>	Month name Jul / July	<code>%A / %a</code>	Weekday Saturday / Sat
<code>%p</code>	AM/PM	<code>%j</code>	Day of year 001-366
<code>%F</code>	<code>= %Y-%m-%d</code>	<code>%T</code>	<code>= %H:%M:%S</code>

Examples on the dataset

```
# Readable timestamp + day-of-week + hour-of-day buckets
index=web sourcetype=access_combined
| eval when = strftime(_time, "%Y-%m-%d %H:%M:%S"),
    dow    = strftime(_time, "%A"),
    hour   = strftime(_time, "%H")
```

```
# Parse a timestamp string coming from a log field into epoch
| makeresults
| eval raw_ts = "18/Jul/2026:09:14:22"
| eval epoch = strptime(raw_ts, "%d/%b/%Y:%H:%M:%S")
```

```
# Age of each event in hours using now()
index=security
| eval age_hours = round((now() - _time) / 3600, 1)
```

```
# Bucket into business vs off hours with relative_time-style logic
index=web
| eval hr = tonumber(strftime(_time, "%H"))
| eval shift = case(hr>=9 AND hr<18, "business",
```

```
hr>=18 AND hr<22, "evening",  
true(), "overnight")
```

NOW() IS FROZEN

`now()` returns the search's dispatch time — the same value for every event. Use `time()` only when you need the true instant of evaluation (rare). For "age" calculations, `now()` is what you want.

14 · String Functions

String functions inspect and reshape text — measuring length, changing case, trimming, replacing with regular expressions, slicing substrings, splitting into multivalue fields, and decoding URLs. They are indispensable for cleaning messy log fields.

Function	Description	Example → result
<code>len(str)</code>	Character count	<code>len("error")</code> → 5
<code>lower(str) / upper(str)</code>	Change case	<code>upper("get")</code> → GET
<code>trim(str,chars)</code>	Strip chars from both ends (default: spaces)	<code>trim("##hi##", "#")</code> → hi
<code>ltrim / rtrim</code>	Trim from left / right only	<code>ltrim(" hi")</code> → "hi"
<code>substr(str,start,len)</code>	Substring; start is 1-based	<code>substr("STRATEGY",1,4)</code> → STRA
<code>replace(str,regex,repl)</code>	Regex replace (all matches)	<code>replace("a1b2","\d","#")</code> → a#b#
<code>split(str,delim)</code>	Split into a multivalue field	<code>split("a,b,c",",")</code> → [a,b,c]
<code>mvjoin(mv,delim)</code>	Join a multivalue field into a string	<code>mvjoin(m,"; ")</code>
<code>mvindex(mv,i,[j])</code>	Pick element(s) from a multivalue field	<code>mvindex(m,0)</code>
<code>mvcount(mv)</code>	Number of values in a multivalue field	<code>mvcount(m)</code>
<code>spath(input,path)</code>	Extract a value from JSON/XML by path	<code>spath(_raw,"user.id")</code>
<code>urldecode(str)</code>	Decode a percent-encoded URL	<code>urldecode("a%20b")</code> → "a b"

Examples on the dataset

```
# Normalize case and measure user-agent length
index=web sourcetype=access_combined
| eval ua_lower = lower(useragent), ua_len = len(useragent)
```

```
# Split a productId like WC-SH-A01 into its parts
index=web
| eval parts = split(productId, "-"),
    family = mvindex(parts, 0),
    variant= mvindex(parts, -1)
```

```
# Extract the JSESSIONID from a URL with a regex replace
index=web
| eval sid = replace(referer, ".*JSESSIONID=([A-Z0-9]+).*", "\1")
```

```
# Decode an encoded query string and grab the host of a referer
index=web
| eval clean_ref = urldecode(referer),
    host_only = mvindex(split(replace(clean_ref,"https?://",""),"/"), 0)
```

SUBSTR IS 1-BASED

Unlike most languages, Splunk's `substr()` starts counting at position 1, not 0. A negative start counts from the end: `substr("STRATEGY",-3)` → "EGY".

REX VS REPLACE

Use `eval replace()` for simple in-place substitutions. For pulling *new named fields* out of raw text, the dedicated `rex` command is clearer: `| rex "JSESSIONID=(?<sid>[A-Z0-9]+)"`.

15 · Comparison & Conditional Functions

These functions make decisions. They branch on conditions, fill in defaults for missing values, test membership, and pattern-match — turning raw values into the categories and flags your analysis needs.

Function	Description
<code>if(cond, a, b)</code>	Returns <code>a</code> if <code>cond</code> is true, else <code>b</code>
<code>case(c1,v1, c2,v2, ...)</code>	Returns the value paired with the first true condition
<code>validate(c1,v1, ...)</code>	Opposite of <code>case</code> : returns the value of the first false condition (good for error messages)
<code>coalesce(a,b,c,...)</code>	Returns the first non-null argument
<code>nullif(a,b)</code>	Returns null if <code>a == b</code> , else <code>a</code>
<code>null()</code>	Returns a null value
<code>in(field, v1, v2, ...)</code>	True if field equals any listed value
<code>like(text, pattern)</code>	SQL-style match; <code>%</code> = any run, <code>_</code> = one char
<code>match(text, regex)</code>	True if the regex matches the text
<code>cidrmatch("cidr", ip)</code>	True if ip falls inside the CIDR subnet
<code>searchmatch("expr")</code>	True if the event matches the given search expression
<code>true() / false()</code>	Boolean literals — <code>true()</code> is the catch-all in <code>case</code>

Examples on the dataset

```
# Classify HTTP status into families with case()
index=web sourcetype=access_combined
| eval status_class = case(
    status>=200 AND status<300, "Success",
    status>=300 AND status<400, "Redirect",
    status>=400 AND status<500, "Client Error",
    status>=500, "Server Error",
    true(), "Unknown")
```

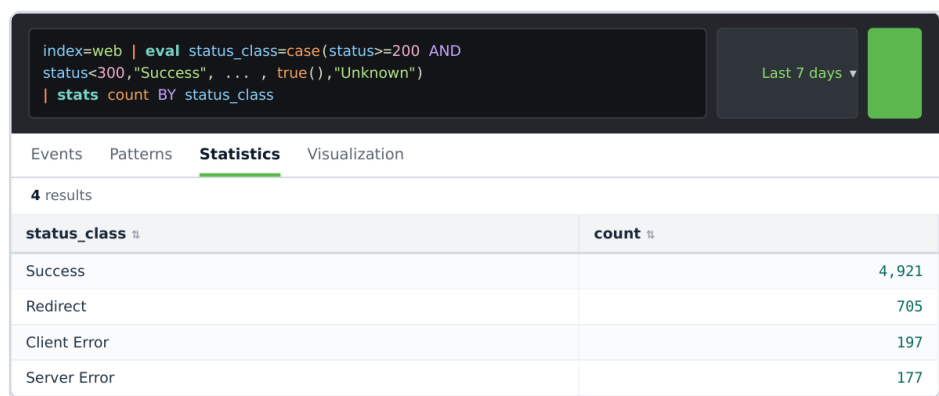


Fig 15.1 — `case()` collapsing raw status codes into families, then counted on the dataset.

```
# coalesce: fill missing product_name from productId
index=web
| eval label = coalesce(product_name, productId, "UNKNOWN")
```

```
# in() and like() for set membership and patterns
index=web
| where in(categoryId, "STRATEGY", "ARCADE", "SHOOTER")
| eval is_wc = if(like(productId, "WC-%"), "yes", "no")
```

```
# cidrmatch: is the source IP internal? (security use case)
index=security
| eval zone = if(cidrmatch("10.0.0.0/8", src_ip), "internal", "external")
```

```
# match: validate an email-ish field with a regex
| eval valid_ip = if(match(clientip, "^\\d{1,3}(\\.\\d{1,3}){3}$"), "ok", "malformed")
```

CASE NEEDS A CATCH-ALL

If no condition matches, `case()` returns null. Always end with `true(), "default"` to guarantee a value. Order matters — the first true condition wins, so put narrow conditions before broad ones.

IF VS CASE

Use `if()` for a single yes/no branch. Reach for `case()` the moment you have three or more outcomes — nesting `if()` quickly becomes unreadable.

16 · Informational Functions

Informational functions answer questions *about* a value rather than transforming it: is it null, is it a number, what type is it? They are the guards you put around calculations to keep dirty data from producing wrong results.

Function	Returns true when...
<code>isnull(X)</code>	X has no value
<code>isnotnull(X)</code>	X has a value
<code>isnum(X)</code>	X is a number
<code>isint(X)</code>	X is an integer
<code>isstr(X)</code>	X is a string
<code>isbool(X)</code>	X is a boolean
<code>typeof(X)</code>	Returns the type name: "Number", "String", "Boolean", "Null", "Invalid"

Examples on the dataset

```
# Data-quality report: how many events are missing price?
index=web action=purchase
| eval price_missing = if(isnull(price), 1, 0)
| stats sum(price_missing) AS missing, count AS total
| eval pct_missing = round(missing/total*100, 2)
```

```
# Only do math on numeric values; label the rest
index=web
| eval bytes_kb = if(isnum(bytes), round(bytes/1024, 2), null())
```

```
# Inspect the actual type of a field while debugging
index=web
| eval t_price = typeof(price), t_status = typeof(status), t_action = typeof(action)
| table price t_price status t_status action t_action
```

GUARD YOUR MATH

Wrapping calculations in `isnum()` or `isnotnull()` prevents a single dirty value from silently dropping a field on that event. In dashboards, this is the difference between a clean panel and mysterious gaps.

17 · Statistical Functions

The **eval** statistical functions operate **within a single event** across a list of arguments or a multivalue field — not across the whole dataset the way the `stats` command does. Use them to pick the largest of several fields, or to inject randomness.

DON'T CONFUSE THE TWO "MIN/MAX"

`stats max(bytes)` finds the largest value *across all events*. `eval biggest = max(m1, m2, m3)` finds the largest of several values *within one event*. Same word, very different scope.

Function	Description	Example → result
<code>max(X, Y, ...)</code>	Largest of the arguments (numbers sort above strings)	<code>max(3, 9, 4) → 9</code>
<code>min(X, Y, ...)</code>	Smallest of the arguments	<code>min(3, 9, 4) → 3</code>
<code>random()</code>	Pseudo-random integer between 0 and 231-1	<code>random() → 1734920581</code>

Examples on the dataset

```
# Pick the peak of several timing fields on the same event
index=web
| eval worst_ms = max(db_ms, render_ms, net_ms)
```

```
# Random sampling: keep roughly 10% of events for a quick look
index=web sourcetype=access_combined
| eval bucket = random() % 10
| where bucket == 0
```

```
# Clamp a value into a range using min + max together
| eval clamped = max(0, min(score, 100))
# keeps score within 0..100
```

FOR DATASET-WIDE STATS

If you want averages, medians, percentiles, standard deviation and the rest *across events*, use the `stats` / `eventstats` / `streamstats` commands from Part 1 — those aggregate functions are far richer than the three eval statistical functions here.

18 · Mathematical Functions

Mathematical functions do arithmetic beyond the basic operators: rounding, powers, roots, logarithms, and constants. They turn raw counts and sizes into rates, ratios and scaled values fit for reporting.

Function	Description	Example → result
<code>abs(X)</code>	Absolute value	<code>abs(-7) → 7</code>
<code>round(X, n)</code>	Round to n decimal places (default 0)	<code>round(3.14159, 2) → 3.14</code>
<code>ceiling(X) / ceil(X)</code>	Round up to next integer	<code>ceiling(2.1) → 3</code>
<code>floor(X)</code>	Round down to previous integer	<code>floor(2.9) → 2</code>
<code>pow(X, Y)</code>	X raised to the power Y	<code>pow(2, 10) → 1024</code>
<code>sqrt(X)</code>	Square root	<code>sqrt(144) → 12</code>
<code>exp(X)</code>	e raised to X	<code>exp(1) → 2.718...</code>
<code>ln(X)</code>	Natural logarithm (base e)	<code>ln(2.718) → ≈1</code>
<code>log(X, base)</code>	Logarithm; base defaults to 10	<code>log(1000) → 3</code>
<code>pi()</code>	The constant π	<code>pi() → 3.14159...</code>
<code>sigfig(X)</code>	Round to significant figures based on input precision	<code>sigfig(1.0034*3) → 3.01</code>
<code>exact(X)</code>	Evaluate with maximum floating-point precision	<code>exact(1/3)</code>

Examples on the dataset

```
# Convert bytes to KB/MB and round for a clean table
index=web sourcetype=access_combined
| eval kb = round(bytes/1024, 2),
      mb = round(bytes/1048576, 4)
```

```
# Conversion rate = purchases / views, as a rounded percentage
index=web
| stats count(eval(action="purchase")) AS buys,
        count(eval(action="view")) AS views BY categoryId
| eval conv_rate = round(buys/views*100, 2)
```

```
# Log scale for a long-tailed byte distribution
index=web
| eval log_bytes = round(log(bytes + 1, 10), 3)
```

```
# Ceiling to bill in whole units; floor to bucket into tiers of 10
| eval billed_units = ceiling(bytes/1000000),
      price_tier    = floor(price/10) * 10
```

ROUND FOR DISPLAY, NOT FOR KEYS

Round values you show to humans, but keep full precision for anything you sort or compare on. Rounding early can collapse distinct values and skew a downstream `dc()` or `sort`.

19 · Cryptographic Functions

Cryptographic functions produce a fixed-length **hash** of a value. In SPL they are used mainly for two things: **pseudonymizing** sensitive fields (so you can group and count without exposing the real value) and generating stable **keys** / **fingerprints** for deduplication and correlation.

Function	Algorithm	Output length
md5(X)	MD5	32 hex chars (128-bit)
sha1(X)	SHA-1	40 hex chars (160-bit)
sha256(X)	SHA-256	64 hex chars (256-bit)
sha512(X)	SHA-512	128 hex chars (512-bit)

Examples on the dataset

```
# Pseudonymize client IPs before sharing a report
index=web sourcetype=access_combined
| eval ip_hash = sha256(clientip)
| stats count, dc(JSESSIONID) AS sessions BY ip_hash
```

```
# Build a stable event fingerprint to detect duplicates
index=web
| eval fingerprint = md5(clientip . "|" . productId . "|" . _time)
| stats count BY fingerprint
| where count > 1
```

```
# Consistent salted hash for correlation across indexes
| eval user_key = sha1("SALT_2026:" . lower(user))
```

SECURITY REALITY CHECK

Hashing is

ONE-WAY BUT NOT SECRET

. For a small value space (like IPv4 addresses) an attacker can brute-force the hash. If pseudonymization must resist reversal, add a secret salt as shown above and keep the salt out of the searchable data. Hashing is not encryption and does not make data compliant on its own.

CHOOSING AN ALGORITHM

Prefer `sha256` for new work. `md5` and `sha1` are fine as fast, non-security fingerprints for dedup/correlation, but avoid them where collision resistance actually matters.

20 · Appendix A — stats vs eventstats vs streamstats

These three commands share the same aggregate functions but differ in what they do to your events. This decision flowchart picks the right one every time.

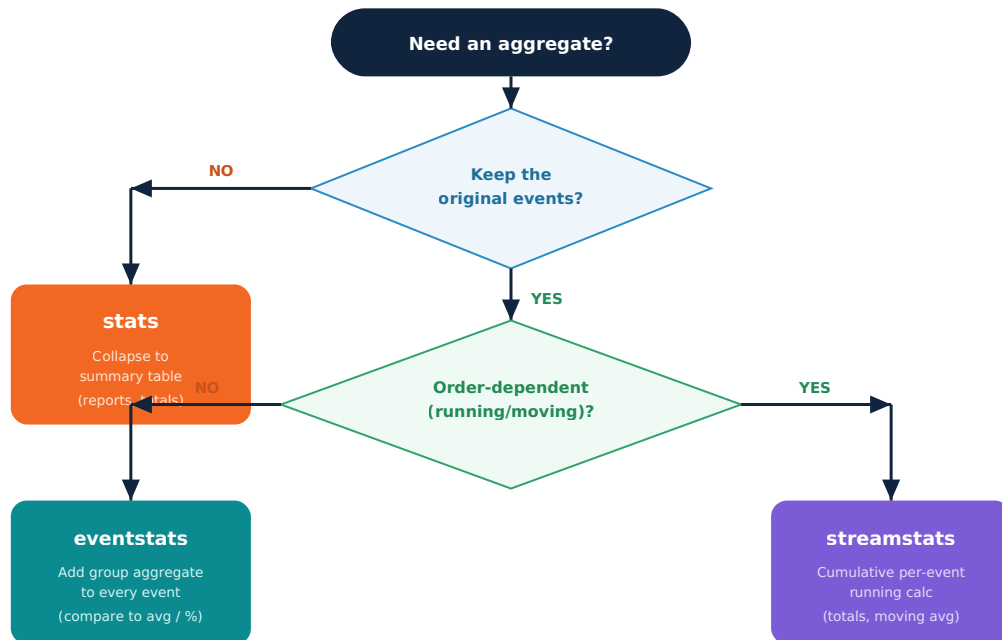


Fig A.1 — Choosing between stats, eventstats and streamstats.

Command	Events kept?	Order matters?	Signature use case
stats	No — collapsed	No	Totals, group summaries, final report tables
eventstats	Yes — all kept	No	Compare each event to its group (above/below average, % of total)
streamstats	Yes — all kept	Yes	Running totals, moving averages, deltas between consecutive events, windowed detection

ONE DATASET, THREE ANSWERS

On `index=web action=purchase` : `stats sum(price) BY categoryId` → revenue per category (3 rows).
`eventstats sum(price) AS total` → every purchase tagged with the grand total so you can compute its share.
`streamstats sum(price) AS running` (after sort by `_time`) → a cumulative revenue curve.

21 · Appendix B — SPL Quick-Reference Cheat Sheet

RETRIEVAL & SCOPE

```
index= sourcetype= source= host=
```

```
| head N · | tail N · | dedup f
```

```
| fields a,b · | fields - _raw
```

```
| tstats count BY sourcetype
```

TIME

```
earliest=-7d@d latest=now
```

```
snap @d @h @w0 @mon
```

```
| bin _time span=1h
```

```
| timechart span=1h count BY action
```

SEARCH DIRECTIVES

```
AND OR NOT ( ) · uppercase
```

```
CASE(x) case-sensitive
```

```
TERM(x) single indexed term
```

```
wildcard term* (avoid *term)
```

AGGREGATION COMMANDS

```
stats f(x) AS a BY g
```

```
eventstats f(x) BY g (keep events)
```

```
streamstats window=N f(x) BY g
```

```
fieldsummary maxvals=10
```

```
appendpipe [ | stats ... ]
```

STATS FUNCTIONS

```
count dc sum avg median mode
```

```
min max range stdev var
```

```
perc95 first last earliest latest
```

```
list values sumsq
```

EVAL · CONVERSION

```
tonumber tostring(x,"commas") printf
```

EVAL · DATE & TIME

```
now() time() relative_time()
```

```
strftime(_time,"%F %T") strptime()
```

EVAL · STRING

```
len lower upper trim ltrim rtrim
```

```
substr replace split mvindex mvjoin
```

```
spath urldecode mvcount
```

EVAL · COMPARISON & CONDITIONAL

```
if case validate coalesce nullif
```

```
in like match cidrmatch searchmatch
```

```
true() false() null()
```

EVAL · INFORMATIONAL

```
isnull isnotnull isnum isint
```

```
isstr isbool typeof
```

EVAL · STATISTICAL

```
max(a,b,...) min(a,b,...) random()
```

EVAL · MATHEMATICAL

```
abs round ceiling floor pow sqrt
```

```
exp ln log pi sigfig exact
```

EVAL · CRYPTOGRAPHIC

```
md5 sha1 sha256 sha512
```

THE ONE-LINE PHILOSOPHY

Filter early (index, time, keyword) → shape per event (eval, rex, where) → aggregate late (stats family) → format for humans (round, tostring, sort). Every good search is a version of this shape.

You've reached the end of the handbook.

Compiled by Vivek Arora · Corporate Trainer, Author & Consultant at StackMaster Tech, Bengaluru. Practise every SPL block against the Buttercup Games tutorial dataset to lock in the muscle memory. For video walkthroughs of these commands, see youtube.com/@StackMasterTech.